HyperText Transfer Protocol (HTTP)

IT5100D AY25/26 S2

NUS National University of Singapore | Computing

© Copyright National University of Singapore. All Rights Reserved.

Application Layer Protocol

Defines:

- Types of messages exchanged
  - E.g., request, response
- Message syntax
  - What fields are in messages
  - How fields are delineated
- Message semantics
  - Meaning of information in fields
- Rules for when and how processes send and respond to messages

Open protocols

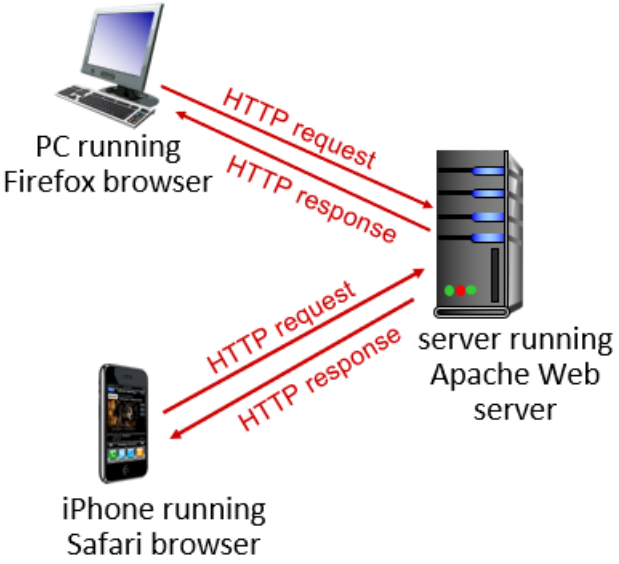
- Defined in publicly accessible RFCs
- Allows for interoperability
- Examples: HTTP, SMTP

Proprietary protocols

- Examples: Skype, Zoom

HTTP Overview

- Web's application-layer protocol
- Client/server model:
  - Client is the web browser that requests, receives (using HTTP), and "displays" Web objects
  - Web server sends (using HTTP) objects in response to requests



HTTP Overview

HTTP uses *TCP*

- Client initiates TCP connection to server, port 80
- Server accepts TCP connection from client
- HTTP messages exchanged between browser (HTTP client) and Web server (HTTP server)
- TCP connection closed

TCP: transport layer protocol, details later

HTTP is *stateless*

- Server maintains no information about past client requests

Aside:

Protocols that maintain "state" are complex!

- Past history (state) must be maintained
- If server/client crashes, their views of "state" may be inconsistent, must be reconciled

© Copyright National University of Singapore. All Rights Reserved.

HTTP Connections: Two Types

Non-persistent HTTP

1. TCP connection opened
2. *At most one object* sent over TCP connection
3. TCP connection closed

Downloading multiple objects required multiple connections.

Persistent HTTP

1. TCP connection opened to a server
2. Multiple objects can be sent over a single TCP connection between client and that server
3. TCP connection closed

© Copyright National University of Singapore. All Rights Reserved.

Non-Persistent HTTP: Example

User enters URL: **www.someSchool.edu/someDepartment/home.index**

(containing text, references to 10 jpeg images)

time

1a. HTTP client initiates TCP connection to HTTP server (process) at www.someSchool.edu on port 80

1b. HTTP server at host www.someSchool.edu waiting for TCP connection at port 80 "accepts" connection, notifying client

2. HTTP client sends HTTP *request message* (containing URL) indicating that client wants object someDepartment/home.index

3. HTTP server receives request message, forms *response message* containing requested object, and sends message into its socket

4. HTTP server closes TCP connection

5. HTTP client receives response message containing html file
Parsing html file, finds 10 referenced jpeg objects

6. Steps 1-5 repeated for each of the 10 jpeg objects

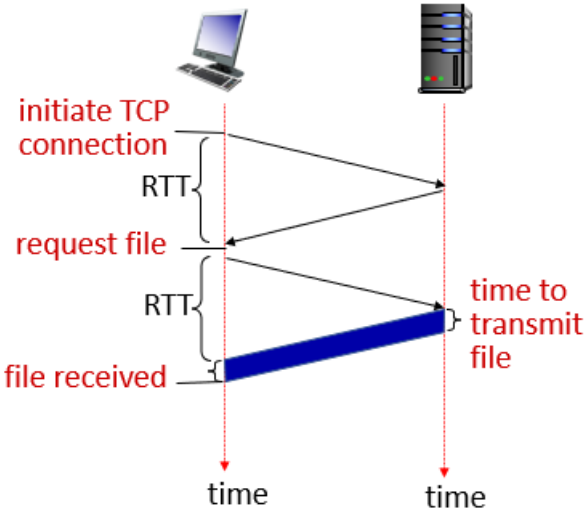
Non-Persistent HTTP: Response Time

Round-trip time (RTT)

- Time for a small packet to travel from client to server and back

HTTP response time (per object)

- One RTT to initiate TCP connection
- One RTT for HTTP request and first few bytes of HTTP response to return
- Object/file transmission time



Non-persistent HTTP response time = 2 RTT + file transmission time

OS overhead for each TCP connection

Connection Management

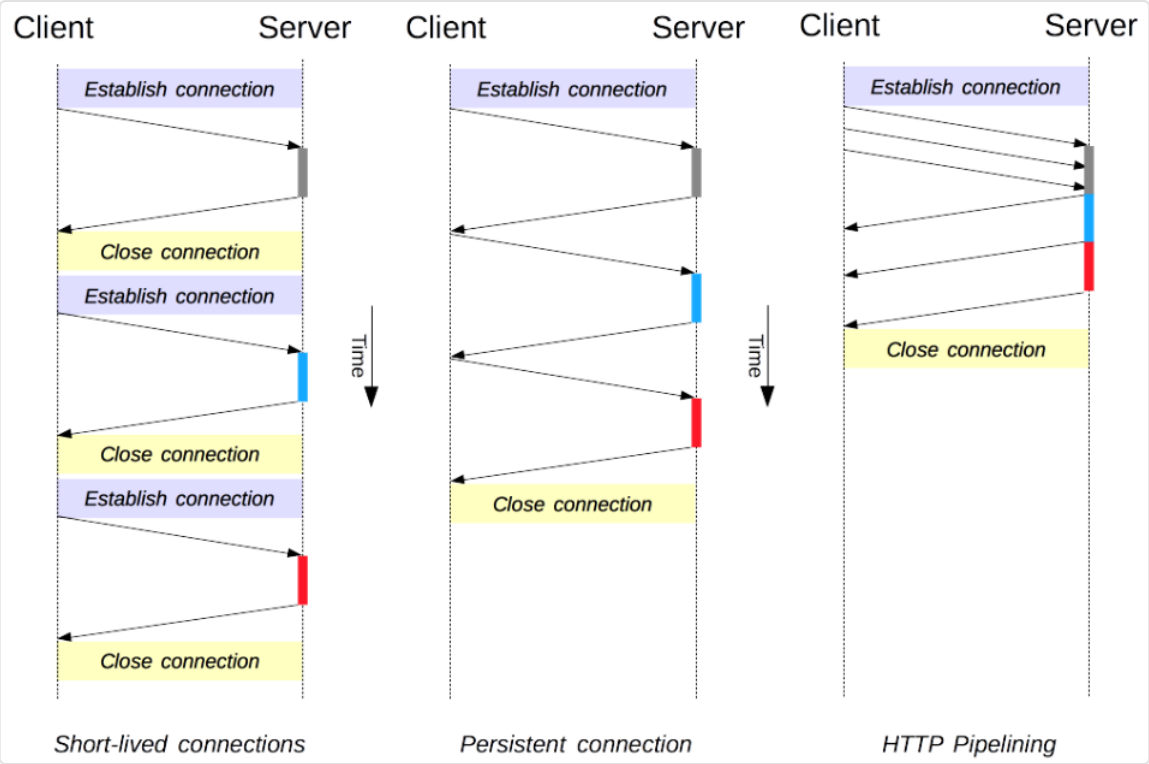


Image source: [MDN Web Docs]

HTTP/1.0

- Non-persistent (short-lived) connection by default
- Set "Connection: keep-alive" to keep connection open

HTTP/1.1

- Persistent connection by default
- Pipelining of requests

HTTP Versions: HTTP/1.1

Key goal: decreased delay in multi-object HTTP requests

HTTP/1.1

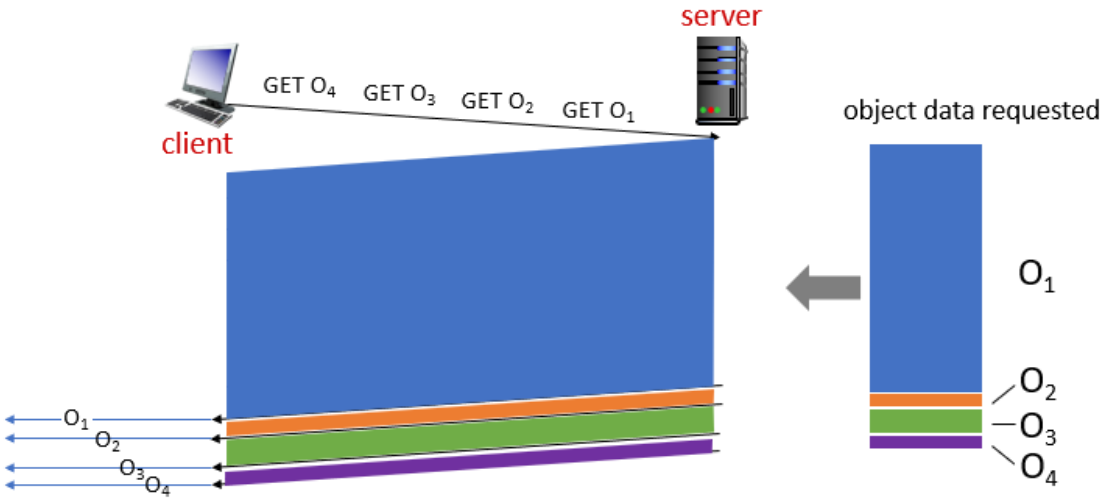
- Introduced *multiple, pipelined* GETs over single TCP connection
- Server responds *in-order* (FCFS: first-come-first-served scheduling) to GET requests
- With FCFS, small object may have to wait for transmission behind large object(s)

→ Head-of-line (HOL) blocking

- Loss recovery (retransmitting lost TCP segments) stalls object transmission

HTTP/1.1 – HOL Blocking

- Client requests 1 large object (e.g., video file) and 3 smaller objects



objects delivered in order requested: O_2, O_3, O_4 wait behind O_1

HTTP Versions: HTTP/2

Key goal: decreased delay in multi-object HTTP requests

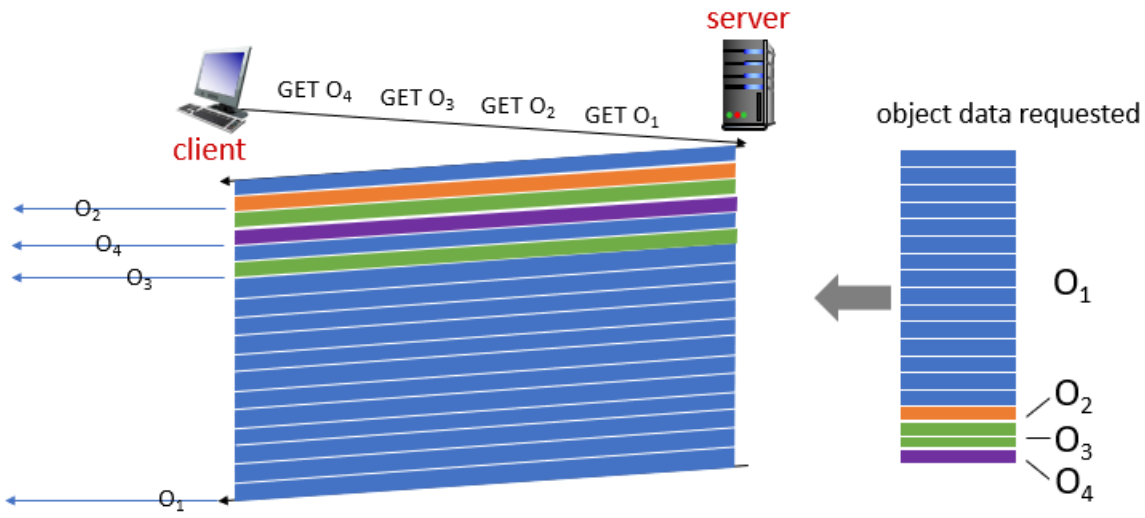
HTTP/2 (RFC 7540, 2015)

Increased flexibility at server in sending objects to client:

- Message format mostly unchanged from HTTP/1.1
- Transmission order of requested objects based on *client-specified object priority* (not necessarily FCFS)
- *Push unrequested objects* to client
- *Divide objects into frames*, schedule frames to mitigate HOL blocking

HTTP/2 – Mitigate HOL Blocking

- Objects divided into frames, with frame transmission interleaved



O_2, O_3, O_4 delivered quickly, O_1 slightly delayed

HTTP/2 to HTTP/3

HTTP/2:

- Over a single TCP connection, recovery from packet loss still stalls all object transmissions
- Does not add security over vanilla TCP connection, although encryption is supported

HTTP/3:

- Uses QUIC, a multiplexed transport layer protocol built on UDP (developed by Google)
- Incorporates Transport Layer Security (TLS) 1.3-based encryption

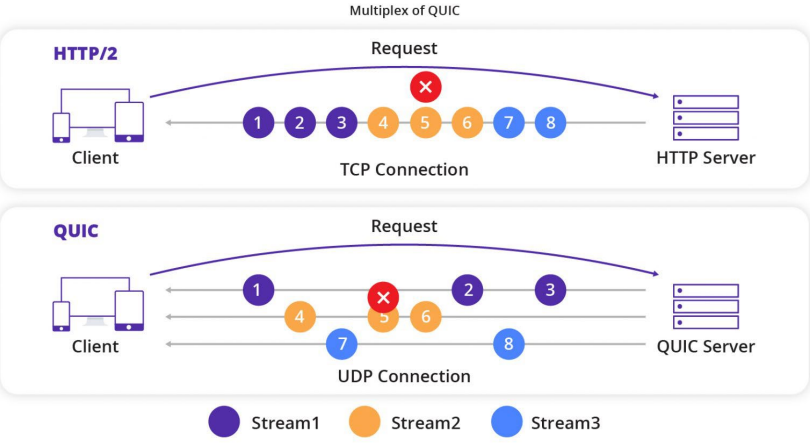


Image source: CDNetworks

HTTP Versions

- HTTP/1.0 (1996, RFC 1945): *Obsolete*
- HTTP/1.1 (1997-2022 last update, RFC 9110)
- HTTP/2 (RFC 9113, Jun 2022)
  - Multiplexing, object prioritizing, server push
- HTTP/3 (RFC 9114, Jun 2022)
  - Security, more pipelining

From RFC 9110:

"[All three major versions of HTTP] have not obsoleted each other because each one has specific benefits and limitations depending on the context of use. Implementations are expected to choose the most appropriate transport and messaging syntax for their particular context."

HTTP/3 challenges:

- Compatibility and interoperability
- Complexity of QUIC, affecting implementation, troubleshooting
- Overhead of QUIC, affecting performance
- Higher resource demands

HTTP/3 Support Discovery

- Servers supporting HTTP/3 typically use HTTP/2 (or HTTP/1.1) as the default and fallback.
- These servers need to signal its HTTP/3 support for clients to use it – two main ways:

Using Alt-Svc header in HTTP response

- First time client connects, server responds with HTTP/2 and includes:

```
Alt-Svc: h3=":443"; ma=86400
```

  - HTTP/3 is available on UDP port 443.
  - This info is valid for 1 day.
- Upon seeing this, client will use HTTP/3 in subsequent requests.

Indicating support in DNS records

- Before first connection, client typically needs to do *DNS lookup* for server's IP address.
- Server can have a DNS record of type HTTPS:

```
example.re 3600 IN HTTPS 1 . alpn="h3,h2"
```

  - "example.re" domain supports HTTP/3 and HTTP/2 in ALPN (Application-Layer Protocol Negotiation).
  - This record has priority 1 (highest) and can be cached for 1 hour.
- With that info, client can directly use HTTP/3 on the first visit.

Browser support for HTTP/3

Adoption of HTTP/2 and HTTP/3

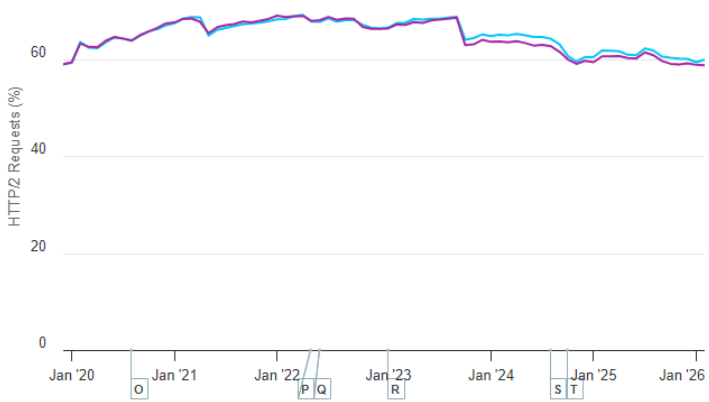
HTTP/2 Requests

The percent of all requests in the crawl using HTTP/2. Note that servers supporting HTTP/2 and HTTP/3 may use HTTP/2 initially due to the way the HTTP Archive starts with a fresh Chrome instance each time, but may use HTTP/3 on subsequent requests.

DESKTOP	MOBILE
59.8%	58.7%
▲1.5%	▼0.3%

Timeseries of HTTP/2 Requests
Source: httparchive.org

Zoom 1m 3m 6m YTD 1y 3y All Dec 2019 → Feb 2026



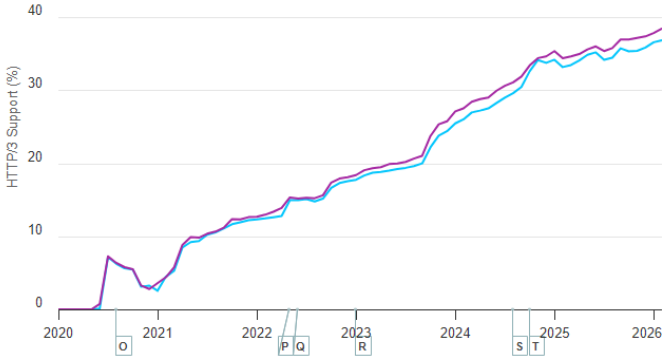
HTTP/3 Support

The percent of all requests in the crawl which support HTTP/3. Note that, due to the way the HTTP Archive starts with a fresh Chrome instance each time, HTTP/3 will often not be used for initial connections until the browser is aware the server supports HTTP/3, which is why we measure support, rather than actual usage like we do for HTTP/2.

DESKTOP	MOBILE
36.8%	38.5%
▲36700.0%	▲5400.0%

Timeseries of HTTP/3 Support
Source: httparchive.org

Zoom 1m 3m 6m YTD 1y 3y All Dec 2019 → Feb 2026



HTTP Request and Response

IT5100D AY25/26 S2

NUS | Computing
National University of Singapore

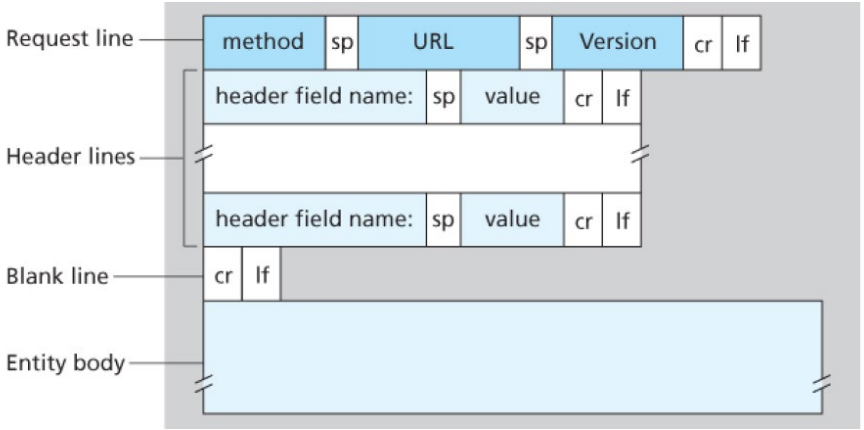
© Copyright National University of Singapore. All Rights Reserved.

HTTP Messages

- Two types of HTTP messages: **request** and **response**
- ASCII (human-readable format)

HTTP Message Syntax

Request

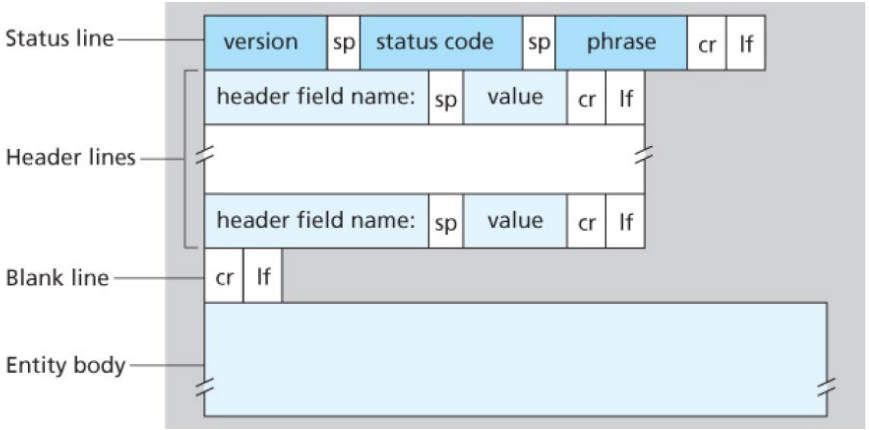



```
GET / HTTP/1.1
Host: developer.mozilla.org
Accept-Language: fr
```

```
POST /test HTTP/1.1
Host: foo.example
Content-Type: application/x-www-form-urlencoded
Content-Length: 27

field1=value1&field2=value2
```

Response



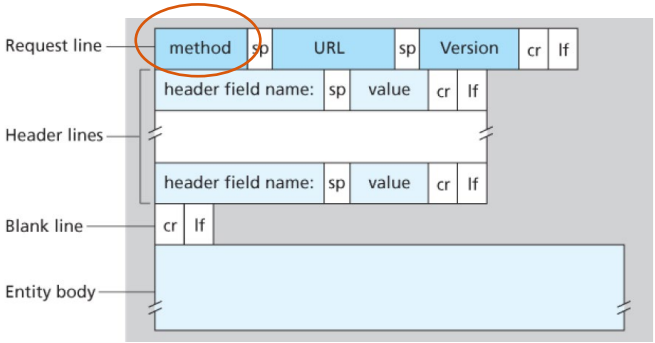

```
HTTP/1.1 200 OK
Date: Sat, 09 Oct 2010 14:28:02 GMT
Server: Apache
Last-Modified: Tue, 01 Dec 2009 20:18:22 GMT
ETag: "51142bc1-7449-479b075b2891b"
Accept-Ranges: bytes
Content-Length: 29769
Content-Type: text/html

<!DOCTYPE html>… (here come the 29769 bytes of the requested web page)
```

HTTP Request Methods

Specification: https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods

- GET
- HEAD
- POST
- PUT
- DELETE
- PATCH
- CONNECT
- OPTIONS
- TRACE



HTTP Request Methods: GET and POST

GET

- Used to request a resource from server
- Limit on URL length depending on server/client/proxy support (usually 8192 bytes)
- Should not have content in the body

→ May include user data in the URL field, following a '?'

E.g.: www.somezoo.com/animalsearch?name=monkey&food=banana

HTTP specification (RFC 9110) "recommends" minimum 8000 octets (8000 bytes)

POST

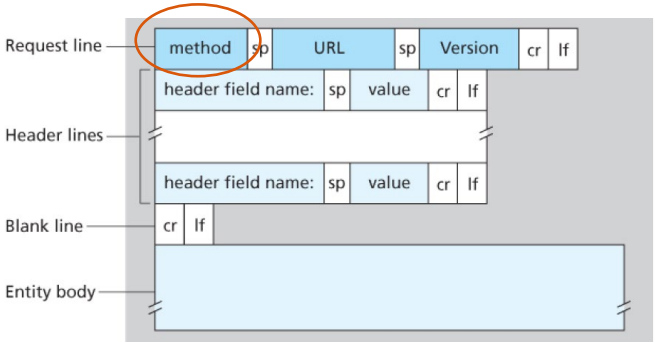
- Used to send user inputs e.g., from a form on Web page
- Input is sent in the entity body

→ Values not visible in the URL

→ No limit on the length of values

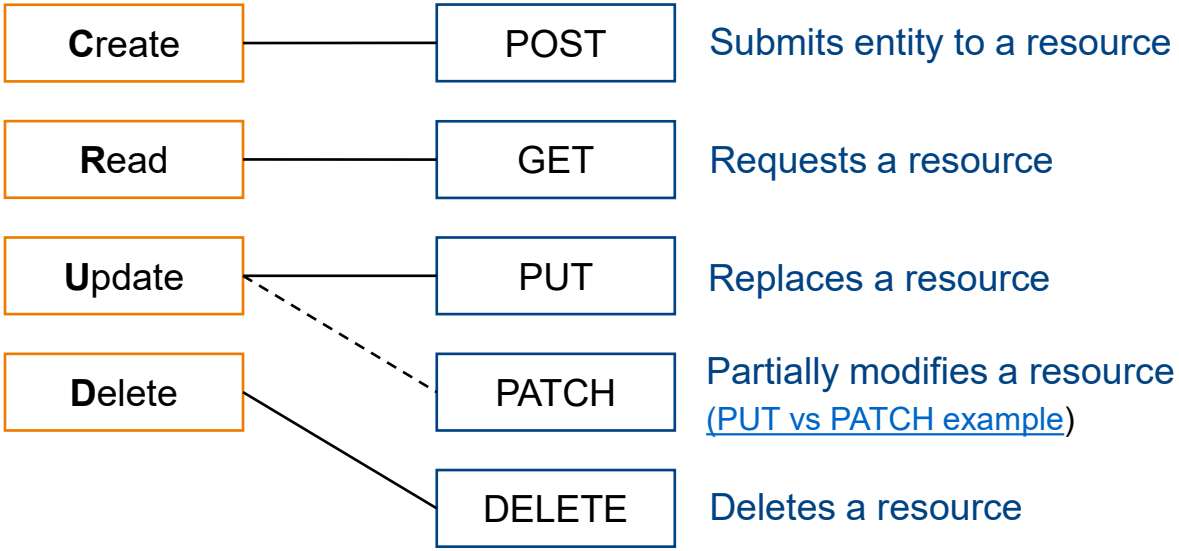
→ Supports different data types

→ Lower performance than GET



HTTP Request Methods and CRUD

CRUD: Four basic operations of persistent storage

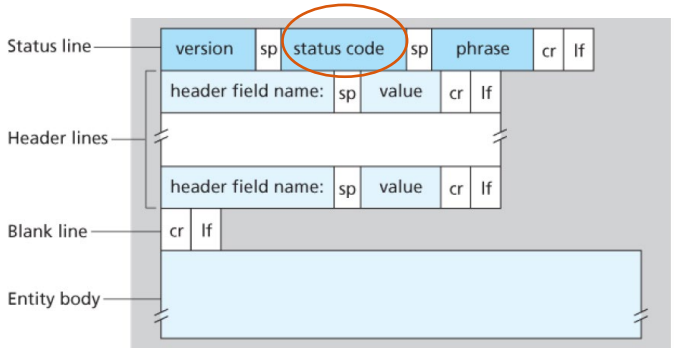


In practice, the actual effect is *whatever is implemented by the backend developers*!

HTTP Response Status Codes

IANA - HTTP Status Code Registry

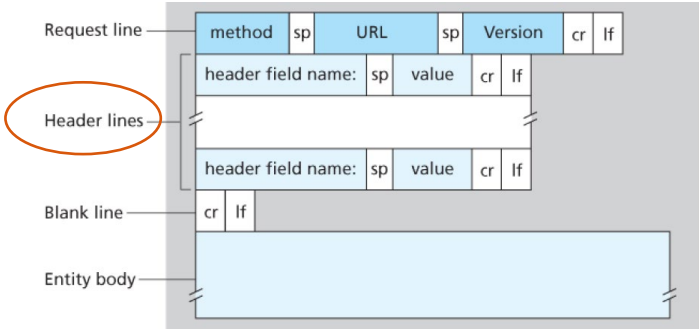
Class	Description	Examples
1xx	Informational	100 Continue 101 Switching Protocols
2xx	Success	200 OK 201 Created
3xx	Redirection	301 Moved Permanently 304 Not Modified
4xx	Client Error	400 Bad Request 401 Unauthorized 403 Forbidden 404 Not Found
5xx	Server Error	500 Internal Server Error 503 Service Unavailable 504 Gateway Timeout



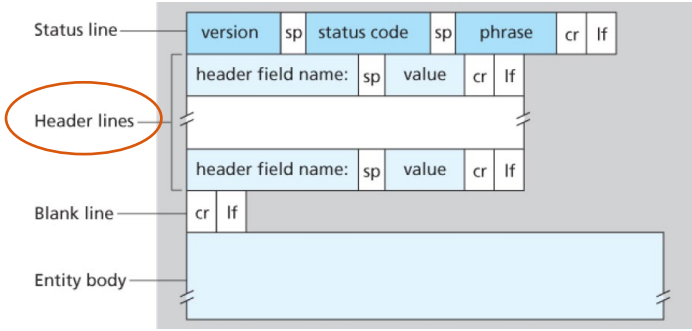
HTTP Message Headers

- Provide info about the request or response
  - Request: specify acceptable formats, provide credentials, specify cache control, etc.
  - Response: info on content length / encoding / last-modified time / …, set cookie, etc.
- [IANA - Registered Message Headers]
- Applications/organizations can add *custom* headers for their own use (e.g., troubleshooting)

Request



Response



Explore with Browser DevTools

DevTools - www.iana.org/assignments/message-headers/message-headers.xml

Elements | Console | Sources | Network | Performance | Memory | Application | Security | Lighthouse | Recorder | Performance insights | AdBlock

Preserve log | Disable cache | No throttling

Filter | Invert | More filters | All | Fetch/XHR | Doc | CSS | JS | Font | Img | Media | Manifest | WS | Wasm | Other

100 ms | 200 ms | 300 ms | 400 ms | 500 ms | 600 ms | 700 ms | 800 ms | 900 ms | 1000 ms | 1100 ms | 1200 ms

Name

message-headers.xml

Headers | Preview | Response | Initiator | Timing

▼ General

Request URL:	https://www.iana.org/assignments/message-headers/message-headers.xml
Request Method:	GET
Status Code:	304 Not Modified
Remote Address:	192.0.33.8:443
Referrer Policy:	strict-origin-when-cross-origin

▼ Response Headers Raw

Cache-Control:	public, s-maxage=1800, max-age=3600
Connection:	Keep-Alive
Content-Security-Policy:	upgrade-insecure-requests; default-src 'self' https://*.iana.org; script-src 'self' 'unsafe-inline' 'unsafe-eval' https://www.google.com https://cse.google.com https://clients1.google.com; style-src 'self' 'unsafe-inline' https://www.google.com; child-src 'self' https://www.youtube.com https://clients1.google.com https://cse.google.com https://www.google.com/; img-src 'self' https://data.iana.org https://www.iana.org https://www.google.com https://www.googleapis.com https://clients1.google.com https://*.gstatic.com;
Cross-Origin-Opener-Policy:	same-origin
Date:	Thu, 23 Jan 2025 09:07:12 GMT
Expires:	Thu, 23 Jan 2025 10:07:12 GMT
Keep-Alive:	timeout=2, max=358
Referrer-Policy:	same-origin
Server:	Apache
Strict-Transport-Security:	max-age=48211200; preload
Vary:	Accept-Encoding
X-Content-Type-Options:	nosniff
X-Content-Type-Options:	nosniff
X-Frame-Options:	DENY

▼ Request Headers Raw

1 / 15 requests | 38.6 kB / 47.1 k

HTTP Cookies

IT5100D AY25/26 S2

NUS | Computing
National University of Singapore

© Copyright National University of Singapore. All Rights Reserved.

Statefulness

Illustration: States in video game control

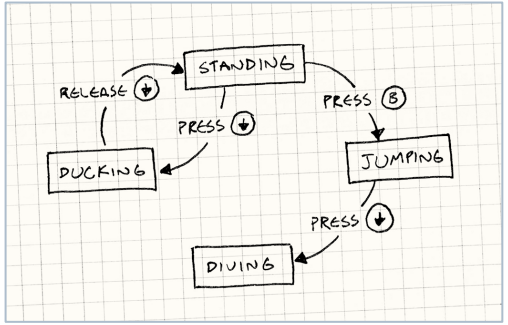
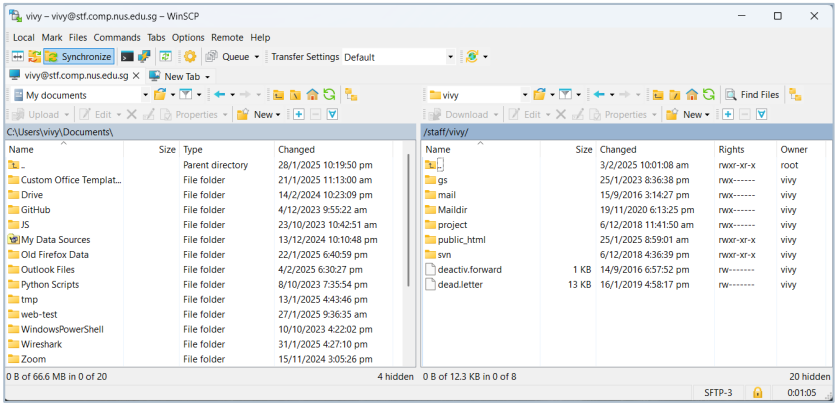


Image source: https://gameprogrammingpatterns.com/state.html

Stateful protocol example: FTP

- Maintains info of current user session, such as current directory



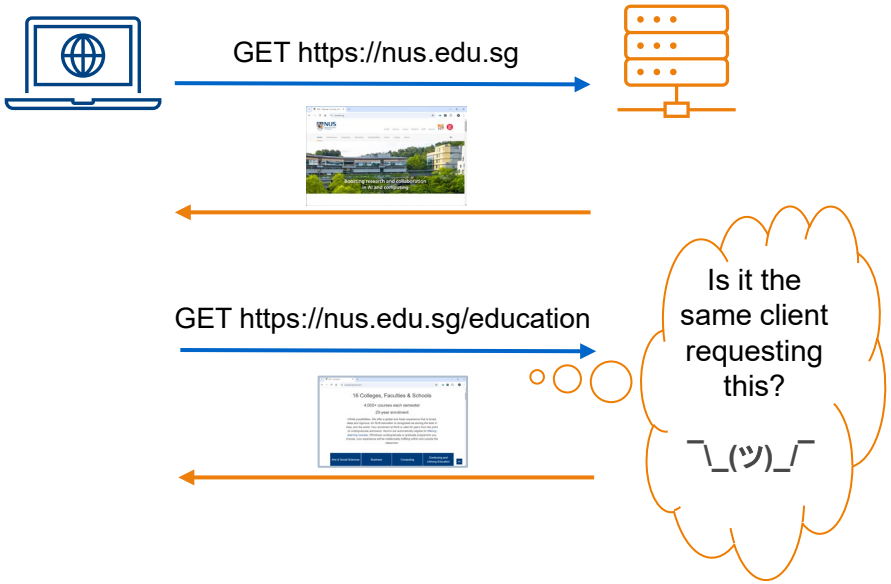
Stateless HTTP

Recall: HTTP server is *stateless*

Once request is answered, it retains no memory of what has taken place

No notion of multi-step exchanges of HTTP messages to complete a Web “transaction”

- No need for client/server to track “state” of multi-step exchange
- All HTTP requests are independent of each other
- No need for client/server to “recover” from a partially-completed-but-never-completely-completed transaction



What if the server wants to “remember” certain things about the client?

→ All desired information must be contained in the request

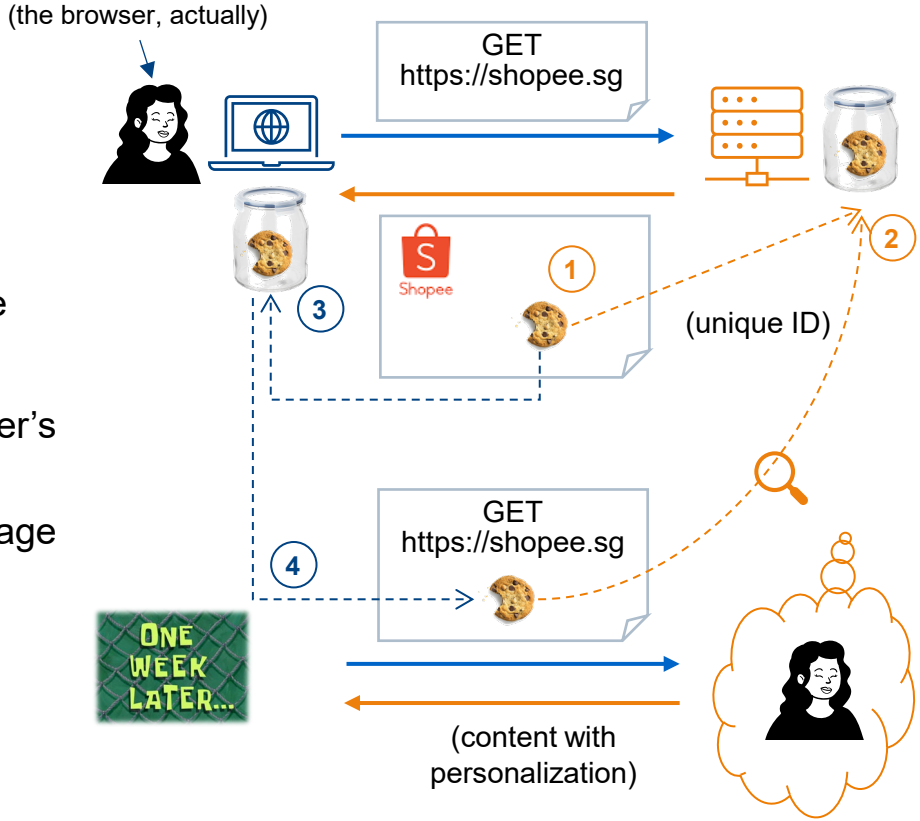
HTTP Cookies

Web sites and client browser use **cookies:** small piece of data, to keep stateful info for the stateless HTTP

Four components:

1. Cookie header line of HTTP response message
2. Entry in database at Web site
3. Cookie file kept on user's host, managed by user's browser
4. Cookie header line in next HTTP request message

Cookies are sent with every HTTP request
→ increasing packet size!



HTTP Cookies – Header Format

Server response (first client request)

```
Set-Cookie: <cookie-name>=<cookie-value>
```

```
HTTP/2.0 200 OK
Content-Type: text/html
Set-Cookie: yummy_cookie=choco
Set-Cookie: tasty_cookie=strawberry

[page content]
```

Client's next request

```
Cookie: <cookie-name>=<cookie-value>
```

```
GET /sample_page.html HTTP/2.0
Host: www.example.org
Cookie: yummy_cookie=choco; tasty_cookie=strawberry
```

Attributes specify desired behaviour e.g., expiration, security

[Set-Cookie on MDN Web Docs]

```
Set-Cookie: <cookie-name>=<cookie-value>
Set-Cookie: <cookie-name>=<cookie-value>; Domain=<domain-value>
Set-Cookie: <cookie-name>=<cookie-value>; Expires=<date>
Set-Cookie: <cookie-name>=<cookie-value>; HttpOnly
Set-Cookie: <cookie-name>=<cookie-value>; Max-Age=<number>
Set-Cookie: <cookie-name>=<cookie-value>; Partitioned
Set-Cookie: <cookie-name>=<cookie-value>; Path=<path-value>
Set-Cookie: <cookie-name>=<cookie-value>; Secure

Set-Cookie: <cookie-name>=<cookie-value>; SameSite=Strict
Set-Cookie: <cookie-name>=<cookie-value>; SameSite=Lax
Set-Cookie: <cookie-name>=<cookie-value>; SameSite=None; Secure

// Multiple attributes are also possible, for example:
Set-Cookie: <cookie-name>=<cookie-value>; Domain=<domain-value>; Secure; HttpOnly
```

HTTP Cookies in the Wild

Headers | Preview | Response | Initiator | Timing | Cookies

Request Cookies ☐ show filtered out request cookies

Name	Value	Domain	Path	Expires / Max-Age	Size	HttpOnly	Secure	SameSite	Partition Key Site	Cross Site	Priority
AMCV_B464317853A9C8390...	-112 ICIDTS%7C201...	.channelnewsas...	/	2026-03-13T02:37:46.855Z	290						Medium
FCNEC	%5E I_hA4oP355HY...	.channelnewsas...	/	2026-02-06T02:37:46.000Z	191						Medium
MeID_Seg	non	.channelnewsas...	/	2026-02-06T02:37:45.000Z	21		✓	None			Medium
_chartbeat2	.173 38809467951....	.channelnewsas...	/	2026-03-08T02:37:47.000Z	88		✓				Medium
_clsk	19u 09466719%7C...	.channelnewsas...	/	2025-02-07T02:37:46.000Z	61						Medium
_ga	GA1 1738809467	.channelnewsas...	/	2026-03-13T02:37:46.807Z	30						Medium
_ga_M37SY0BV84	GS1 4.1.173880955...	.channelnewsas...	/	2026-03-13T02:39:12.419Z	52						Medium
_hjSession_3031977	eyJp NTBmLTdiZGQt...	.channelnewsas...	/	2025-02-06T03:08:17.000Z	158		✓	None			Medium
adtechTargetingKeys	non	.channelnewsas...	/	2026-02-06T02:37:45.000Z	33		✓	None			Medium
ga_pageviews	1	www.channelne...	/	2025-02-06T03:07:46.000Z	13						Medium
reese84	3:Pc CBYrThER3w==...	.www.channeln...	/	2025-03-08T02:37:48.175Z	551			Lax			Medium
s_nr	173 peat	.channelnewsas...	/	2025-03-08T02:37:45.000Z	24						Medium
s_tp	107	.channelnewsas...	/	Session	9						Medium

Redacted

Headers | Payload | Preview | Response | Initiator | Timing | Cookies

Request Cookies ☐ show filtered out request cookies

Name	Value	Domain	Path	Expires / Max-Age	Size	HttpOnly	Secure	SameSite	Partition Key Site	Cross Site	Priority
test_cookie	CheckForPermission	.doubleclick.net	/	2025-02-06T02:56:34.184Z	29		✓	None			Medium

This site has cookies in another partition, that were not sent with this request. Learn more

Response Cookies

Name	Value	Domain	Path	Expires / Max-Age	Size	HttpOnly	Secure	SameSite	Partition Key Site	Cross Site	Priority
IDE	AHWqTUIKR-Q2ZE_CIHL3PU2JNFk...	.doubleclick.net	/	2027-02-06T02:41:43.000Z	173	✓	✓	none			Medium
test_cookie		.doubleclick.net	/	2008-08-01T22:45:55.000Z	108		✓	none			Medium

HTTP Cookies

Main usage

- Session management: logged in users, shopping cart (now uncommon)
- Personalization (user preferences)
- Tracking user's browsing activity (more precision than using IP address)

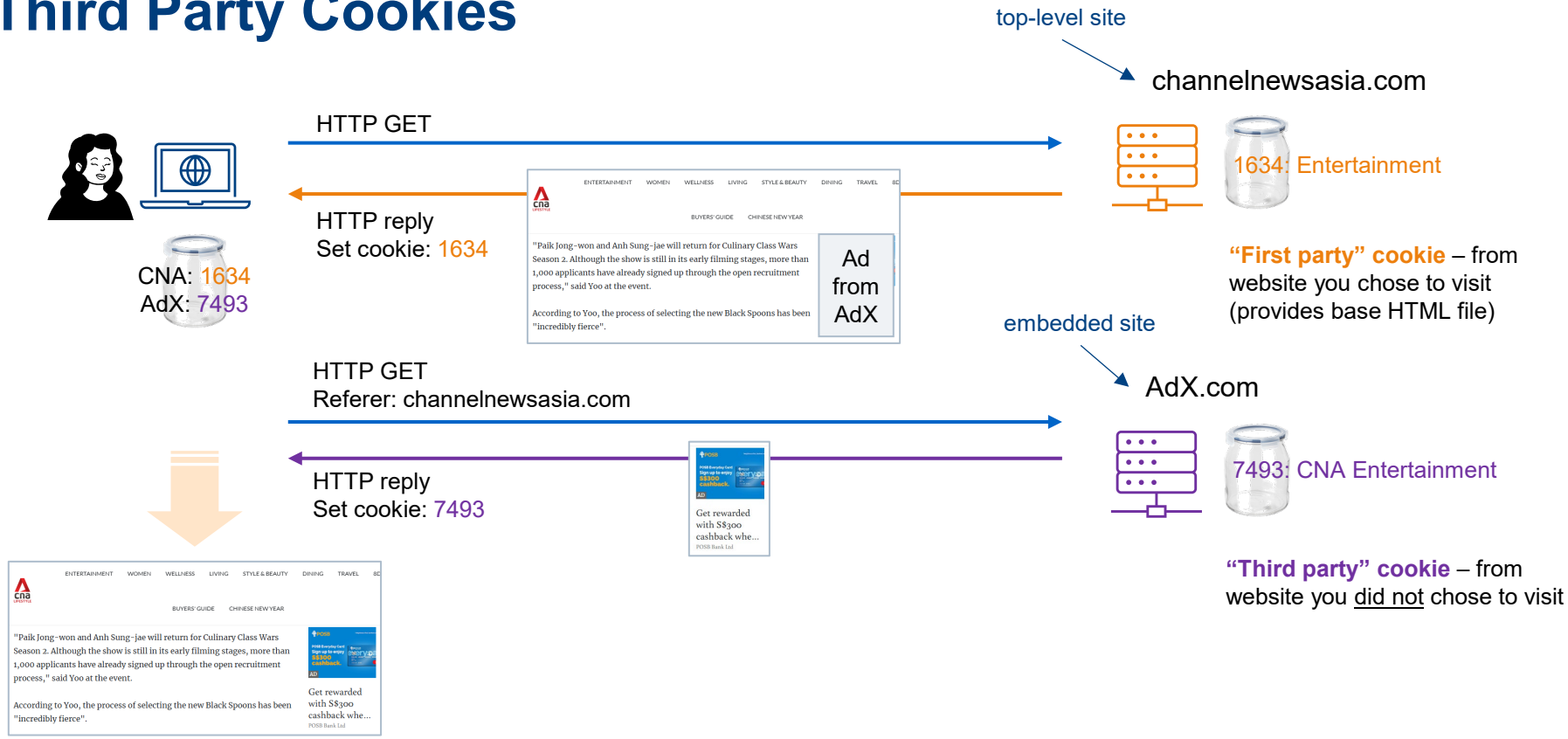
Main types

- Session cookies (or transient cookies): Used only while navigating the website; deleted when browser is closed
- Persistent cookies: Remains until expiry (indefinite if not set!), typically for authentication or tracking

Privacy implications

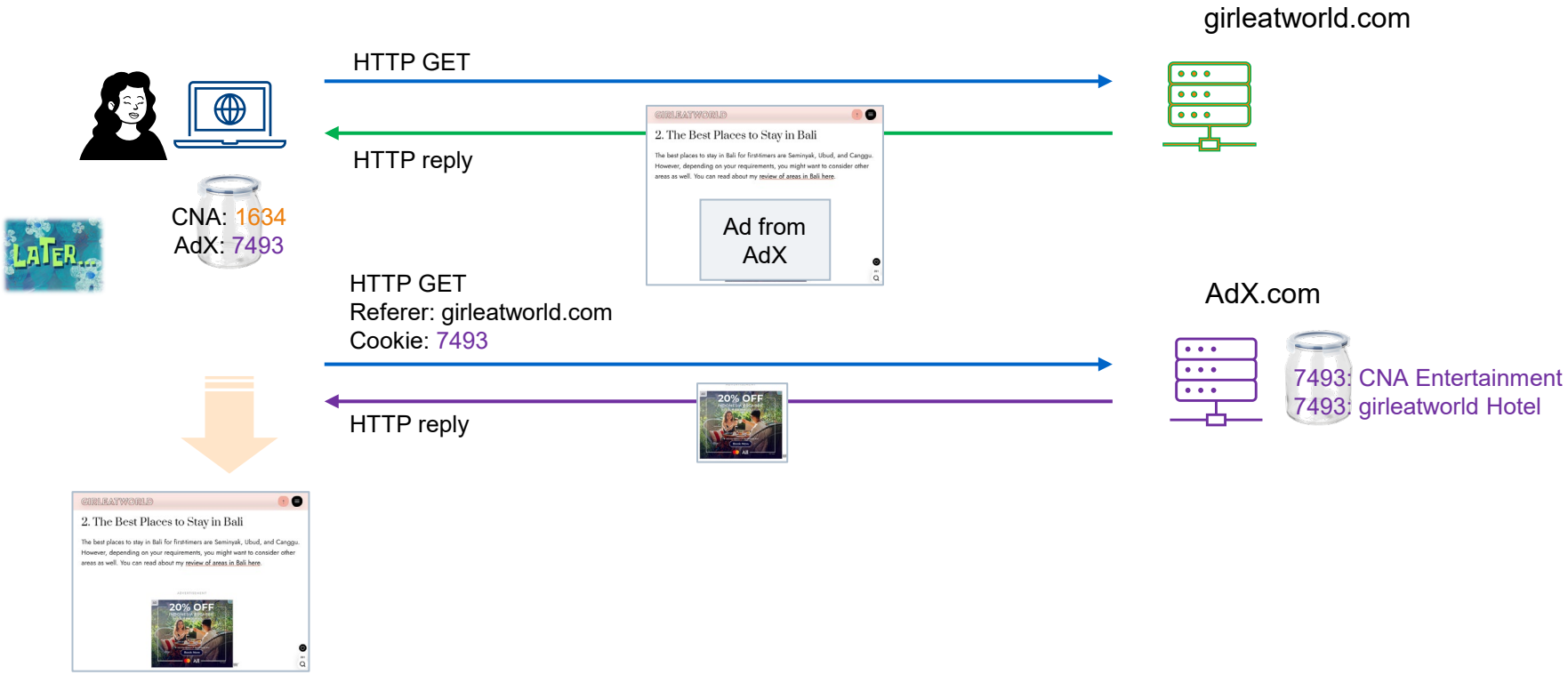
- Sites can learn a lot about your browsing habits via cookies
- Third party persistent cookies (tracking cookies) allow common identity (cookie value) to be tracked across multiple web sites

Third Party Cookies



CNA page with embedded AdX ad displayed

Third Party Cookies – Tracking Browsing History

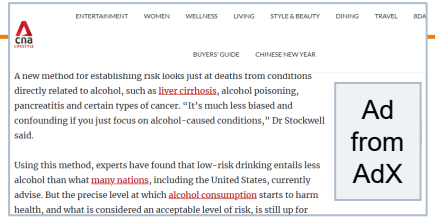


Third Party Cookies – Tracking Browsing History

CNA: 1634
AdX: 7493

HTTP GET
Cookie: 1634

HTTP reply



channelnewsasia.com

1634: Entertainment
1634: Health

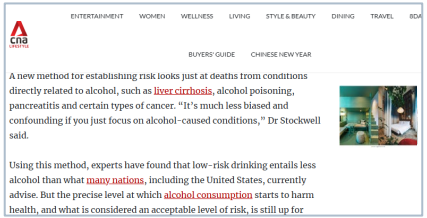
HTTP GET
Referer: channelnewsasia.com
Cookie: 7493

HTTP reply

Returns ad for hotels!

AdX.com

7493: CNA Entertainment
7493: girleatworld Hotel
7493: CNA Health

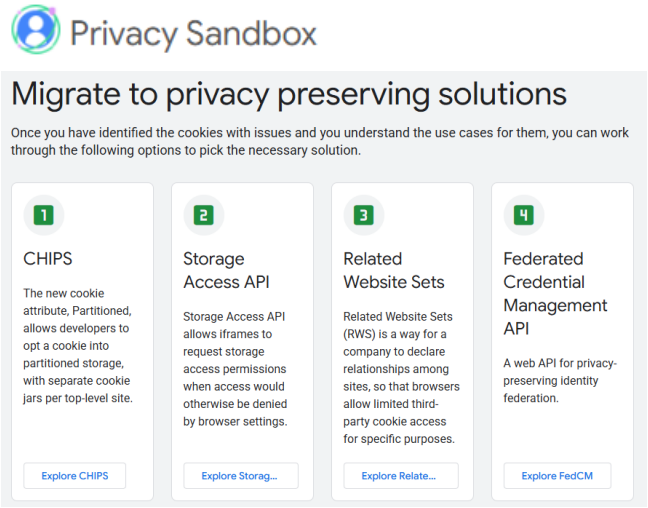


Tracks user's web browsing over sites with AdX ads, can return targeted ads based on browsing history

Third Party Cookies

Third party tracking via cookies:

- Disabled by default in Firefox, Safari browsers
- Chrome browser: phase-out (as of Jan 2025), with possibility to apply for “grace period” and “temporary exceptions”



Cookie Limits

- How small is “small”?
  - Specification (RFC 6265):

In practice, different limits *depending on implementation by different browsers*!

```
6.1.  Limits

   Practical user agent implementations have limits on the number and
   size of cookies that they can store.  General-use user agents SHOULD
   provide each of the following minimum capabilities:

   o  At least 4096 bytes per cookie (as measured by the sum of the
      length of the cookie's name, value, and attributes).

   o  At least 50 cookies per domain.

   o  At least 3000 cookies total.

   Servers SHOULD use as few and as small cookies as possible to avoid
   reaching these implementation limits and to minimize network
   bandwidth due to the Cookie header being included in every request.

   Servers SHOULD gracefully degrade if the user agent fails to return
   one or more cookies in the Cookie header because the user agent might
   evict any cookie at any time on orders from the user.
```

Cookies and Storage

- Cookies were once used for general client-side storage, but this is discouraged now
  - Certain data may be stored that are not meant to be sent to server

- Modern storage APIs are recommended instead
- E.g., Web Storage API (string-only key/value pairs)
  - **sessionStorage**: persists until browser (or tab) is closed
  - **localStorage**: persists until cleared (unless in private/incognito mode)
- Other technologies exist

HTTP Caching

IT5100D AY25/26 S2

NUS | Computing
National University of Singapore

© Copyright National University of Singapore. All Rights Reserved.

HTTP Caching

Principle: Store the copy of a response associated with a request, to reuse for subsequent requests

- Faster response to client
- Lower load on server

RFC 9111 defined two main types:

- Private caches: tied to the specific client (typically a browser cache)
- Shared caches: serve multiple users, located between client and server

Browser Cache

DevTools - developer.chrome.com/docs/devtools/storage/cache

Elements | Console | Sources | Network | Performance | Memory | Application | Security | Lighthouse | Recorder | AdBlock

Preserve log | Disable cache | No throttling

Filter | Invert | More filters | All | Fetch/XHR | Doc | CSS | JS | Font | Img | Media | Manifest | WS | Wasm | Other

100,000 ms | 200,000 ms | 300,000 ms | 400,000 ms | 500,000 ms | 600,000 ms | 700,000 ms

Name	Method	Status	Protocol	Type	Initiator	Size	Time
log_event?alt=json	POST	200	h3	xhr	www-embed	50 B	
icon-144x144.png	GET	200	h3	fetch	sw.js:21	(disk cache)	
manifest.json	GET	200	h3	fetch	sw.js:22	(disk cache)	
web-vitals.attribution.iife.js	GET	200	h2	fetch	web-vitals.at	(disk cache)	
web-vitals.attribution.iife.js	GET	302	h2	fetch /...	sw.js:25	411 B	
app.css	GET	200	h2	fetch	sw.js:21	(disk cache)	

Served from disk cache, resource size: 14.2 kB

DevTools - developer.chrome.com/docs/devtools/storage/cache

Elements | Console | Sources | Network | Performance | Memory | Application | Security

Application
- Manifest
- Service workers
- Storage

Storage
- Local storage
- Session storage
- Extension storage
- IndexedDB
- Cookies
- Private state tokens
- Interest groups
- Shared storage
- Cache storage
  - devsite.pwa_RUNTIME...
- Storage buckets

Background services
- Back/forward cache
- Background fetch
- Background sync
- Bounce tracking mitigati...
- Notifications
- Payment handler
- Periodic background sync
- Speculative loads
- Push messaging
- Reporting API

Frames
- top

Filter by path

https://developer.chrome.com

Origin: https://developer.chrome.com

Bucket name: default

Is persistent: No

Durability: relaxed

Quota: 0 B

Expiration: None

#	Name	Response-Type	Content-Type	Content-Length	Time Cached	Vary Header
0	/_pwa/chrome/icons/icon-144x144.png	basic	image/png	14,193	2/6/2025, 12:14:47 PM	
1	/_pwa/chrome/manifest.json	basic	application/json	1,063	2/6/2025, 12:14:46 PM	
2	/_static/offline?partial=1&hl=en	basic	application/json	10,870	2/6/2025, 12:14:47 PM	Cookie, Accept-Encoding
3	/blog/inside-browser-part1?partial=1&hl=en	basic	application/json	21,954	1/23/2025, 12:05:41 AM	Cookie, Accept-Encoding
4	/blog/removing-push?partial=1&hl=en	basic	application/json	18,700	2/4/2025, 4:29:55 PM	Cookie, Accept-Encoding
5	/docs/devtools/network/reference/?hl=en	basic	text/html	46,131	1/24/2025, 4:39:35 PM	Cookie, Accept-Encoding
6	/docs/devtools/network/reference/?partial=1&hl=en	basic	application/json	49,738	1/31/2025, 11:44:44 AM	Cookie, Accept-Encoding
7	/docs/devtools/network?partial=1&hl=en	basic	application/json	35,890	2/6/2025, 12:15:23 PM	Cookie, Accept-Encoding
8	/docs/devtools/storage/cache?hl=en	basic	text/html	27,393	2/6/2025, 12:15:16 PM	Cookie, Accept-Encoding
9	/docs/devtools/storage/cache?partial=1&hl=en	basic	application/json	29,658	2/6/2025, 12:14:47 PM	Cookie, Accept-Encoding
10	/extras.css	basic	text/css	246	2/6/2025, 12:14:46 PM	Cookie, Accept-Encoding

Headers | Preview

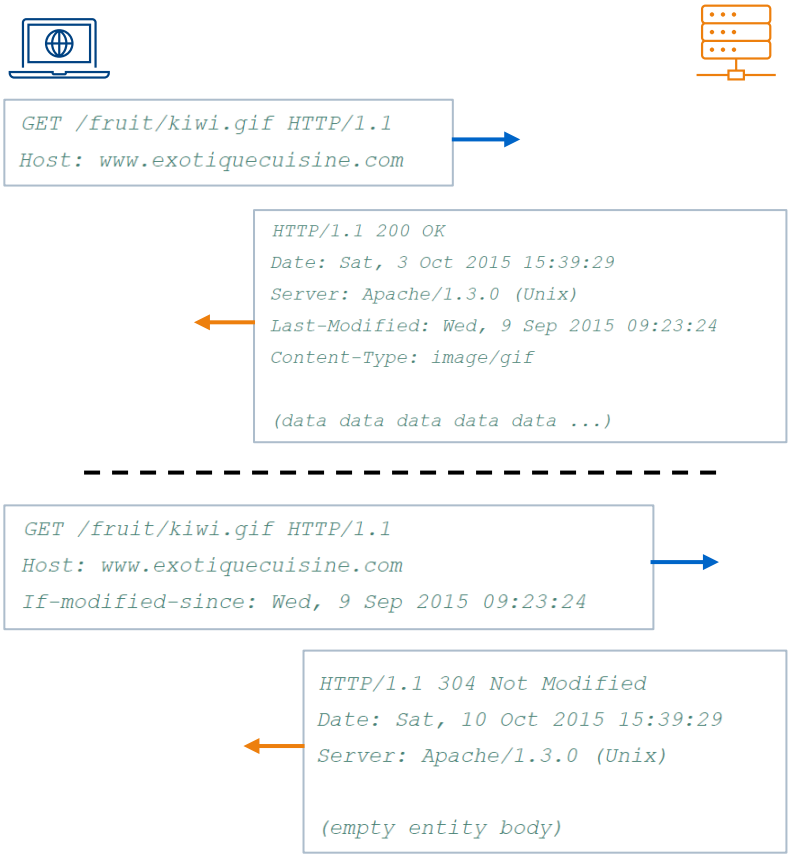
14.2 kB | 144 × 144 | 1:1 | image/png

Total entries: 306

HTTP Mechanism: Conditional GET

To ensure: Reuse only if the copy of response in cache is still up-to-date

- When sending object, server response includes a header specifying when the object is last modified at server
  - `Last-Modified: <timestamp>`
- When requesting object, client request includes a header specifying the last-modified timestamp of the copy in its cache
  - `If-modified-since: <timestamp>`
- Server response contains no object if client-cached copy is up-to-date:
  - `HTTP/1.0 304 Not Modified`



Cache-Control

Response Directives: server response header to tell cache what caching is allowable for the object

- `Cache-Control: max-age=<seconds>`
- `Cache-Control: no-cache`
- `Cache-Control: no-store`
- `Cache-Control: private`
- `Cache-Control: immutable`
- …

Request Directives: client request header to tell cache how to handle caching for the request

- `Cache-Control: max-age=<seconds>`
- `Cache-Control: no-cache`
- `Cache-Control: no-store`
- `Cache-Control: only-if-cached`
- …

Read the specification at MDN Web Docs

Web Cache (or Proxy Server)

A shared cache that satisfies client requests on behalf of an origin Web server

- Has storage to keep copies of recently requested objects

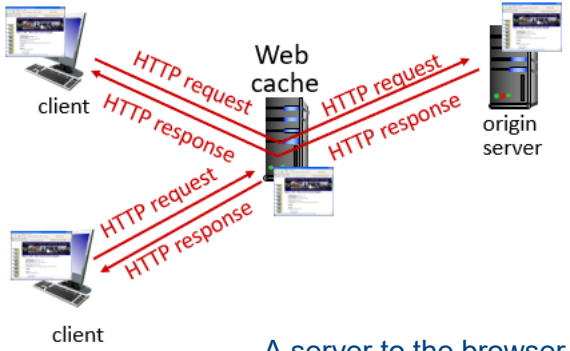
HTTP request from browser is first directed to the Web cache

If the Web cache has the object in storage:

- It checks if the object is up-to-date by sending a request to the origin server
- It sends the object to the client

If no:

- It requests the object from the origin server
- It keeps a copy of the object, and sends the object to the client



A server to the browser
+
A client to the origin server

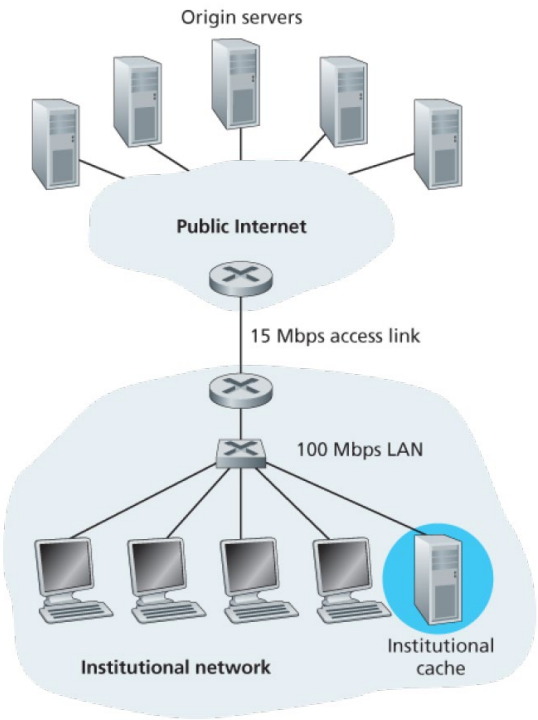
Web Cache

May be installed by CDNs, institutions

- Configure the end systems in their networks to point to the proxy server address
- With HTTPS, caching is only applicable if proxies are allowed to decrypt content

Benefits:

- For the client: reduced response time for requests
  - Web cache is usually closer and has high-speed connection to the clients
- For the institution: reduced traffic on its access link to the Internet
  - Lower bandwidth to support
- For the Internet overall: reduced Web traffic → generally better performance for all applications



Related Concepts

Server caching

- Store frequently requested data to save resources from re-generating or re-fetching from database / third-party API
- Important to manage validity of data in cache

Reverse proxy server

- An intermediary to forward client requests to one or more (internal) web servers
- Appears to clients to be an “ordinary” web server
- Reasons to use:
  - Load balancing among multiple internal servers
  - Avoid directly exposing internal servers to the internet
  - Possible features e.g., security (firewall, encryption), performance (caching, compression)